

EMR-on-EKS Streaming POC — Run Failure Report

Prepared 2026-05-15 · Run 000000037h6oker0s23 · v3.1 script + R-series Spot executors

Bottom line. The run failed at **10 h 8 m** with the same driver-eviction signature as the previous 29 h run — driver pod received SIGTERM (exit code 143), Spark shutdown hooks ran cleanly, no application exception. EMR reports `USER_ERROR` with detail “*Job Driver pod was killed, evicted or preempted.*”. The silver-cadence fix held (no metadata-bloat curve, no MERGE planning ramp); the new failure surface is executor churn on Spot — three executor decommissions in 10 h, each costing the driver a 2.5-minute batch stall. The Spark UI is now reachable for this run — screenshots are the source of the executor snapshot below.

Run snapshot

Job-run ID	000000037h6oker0s23
Job name	iot-streaming-bronze-silver
Script version	v3.1 + silver MERGE every 10 bronze batches (cadence reduction)
Submission JSON	job-submit-streaming-rspot.json (R-series, executors on Spot, driver on-demand)
Submitted	2026-05-14 13:55:13 UTC (19:25 IST)
Finished	2026-05-15 00:03:22 UTC (05:33 IST)
Elapsed	10 h 8 m
State	FAILED — failureReason <code>USER_ERROR</code>
stateDetails	<i>Job Driver pod was killed, evicted or preempted.</i>
Driver node	ip-10-70-19-11.ap-south-1.compute.internal
Driver pod exit	container spark-kubernetes-driver terminated at 2026-05-15T00:02:45Z, exit code 143 (SIGTERM), reason Error

Run-history comparison

Run	Duration	Outcome	Cause
...fsjcis8rer	19 h 6 m	Failed	Executor evictions + Fluentd-OOM blind-spot
...g74dd0ivkf	7 h 44 m	Failed	Same Fluentd-OOM blind-spot
...glvqikoonn	3 h 9 m	Failed	Driver pod SIGTERMed (Karpenter consolidation)
...gncv3v0996	4 h 33 m	Failed	Executor graceful decommission (Karpenter)
...grqasb01cr (v3)	2 h 12 m	Failed	S3 Tables maintenance vs Iceberg MERGE — commit conflict

Run	Duration	Outcome	Cause
...gscnp2qanr (v3.1)	29 h	Failed	Iceberg silver-MERGE metadata bloat → driver SIGTERM (evicted)
...h6oker0s23 (v3.1 + R-spot)	10 h 8 m	Failed	3 Spot exec decommissions + driver SIGTERM (same eviction signature)

Driver termination — the smoking gun

From the JobController control-log (control-logs/000000037h6oker0s23-rjgmm/stderr.gz) at 2026-05-15 00:03:04 UTC, immediately after the driver pod transitioned to phase: Failed:

```
pod name: spark-000000037h6oker0s23-driver
namespace: emr-prod
node name: ip-10-70-19-11.ap-south-1.compute.internal
phase: Failed
container: spark-kubernetes-driver
container state: terminated
container started at: 2026-05-14T13:55:31Z
container finished at: 2026-05-15T00:02:45Z
exit code: 143
termination reason: Error
```

Exit 143 = 128 + 15 = SIGTERM. Critically, the driver-stderr ends with "Kubernetes client has been closed" followed by clean s3a metrics-system shutdown — these are Spark shutdown-hook lines, which run only when the JVM receives SIGTERM, not SIGKILL. If kubelet had OOMKilled the pod (exit 137 / SIGKILL), the JVM would have died mid-instruction with no hook output. So the driver was **terminated externally** while still healthy enough to drain. Same signature as run ...gscnp2qanr; this is not a regression on a known-fixed failure mode, it is the *same* unsolved failure mode firing 19 h sooner.

Failure timeline (UTC)

Time	Event
13:55:13	Job-run created (EMR API)
13:55:31	Driver pod scheduled on ip-10-70-19-11
13:55:46	SparkContext up (Spark 3.5.5-amzn-0 / Java 17.0.18)
13:57:07	First <i>falling behind</i> WARN — spent 46 777 ms (cold-start, expected)
14:08:19	Executor 4 decommissioned — ~13 min lifetime, no blocks to migrate (Spot reclaim moments after pod ready)
~14:00–20:00	Steady state: per-batch spend 30–35 s, bronze fires every batch, silver every 10th
20:09:41	Executor 1 decommissioned (~6 h 14 m lifetime); clean drain, all blocks migrated
20:12:44	Next batch spent 164 687 ms (~2 m 45 s) — backlog from exec-1 loss
21:25:37	Executor 2 decommissioned (~7 h 30 m); clean drain
21:28:45	Next batch spent 165 370 ms (~2 m 45 s) — backlog from exec-2 loss

Time	Event
23:59:36	Bronze Batch 1278 written
00:00:53	Silver Batch 1280 upserted (last silver MERGE in the run)
00:02:41	Bronze Batch 1283 written (last successful batch)
00:02:42	Driver stderr last line: <code>Kubernetes client has been closed → SIGTERM received, shutdown hooks running</code>
00:02:45	Driver container terminated, exit 143
00:03:04	JobController observes pod phase: Failed
00:03:22	EMR job-run state → FAILED

Spark UI snapshot (Executors page)

The Spark UI was reachable for this run — a first since the AWS Support ticket was raised. Two screenshots saved as `Media.jpeg` (Stages overview) and `Media (2).jpeg` (Executors page).

Executors summary table

Group	Storage Mem	Disk	Cores	Complete Tasks	Task Time (GC)	Input	Shuffle R / W
Active (2)	422.5 MiB / 9.7 GiB	0 B	4	17 797	11.3 h (17 s)	2.6 GiB	5.9 / 5.9 GiB
Dead (3)	249.3 MiB / 16.7 GiB	0 B	12	51 241	3.6 h (1.1 min)	12.7 GiB	29.9 / 29.9 GiB
Total (5)	671.8 MiB / 26.4 GiB	0 B	16	69 038	14.9 h (1.4 min)	15.2 GiB	35.8 / 35.8 GiB

Per-executor

ID	Address	Status	Peak JVM OnHeap	Peak Storage	Cores	Failed Tasks
driver	spark-...-driver-svc:7079	Active	—	—	0	0
1	10.70.31.49:43767	Dead	7.8 GiB / 273.9 MiB	80.7 MiB / 0.0 B	4	0
2	10.70.22.93:33065	Dead	0.0 B / 0.0 B	0.0 B / 0.0 B	4	0
3	10.70.19.147:42697	Active	5.1 GiB / 268 MiB	9.9 MiB / 0.0 B	4	0
4	10.70.24.54:43859	Dead	0.0 B / 0.0 B	0.0 B / 0.0 B	4	0

Reading the table. Exec-2 and Exec-4 show all-zero peak metrics — they were Spot-reclaimed before they accumulated any meaningful workload (exec-4 lived ~14 s). Exec-1 carried real load (7.8 GiB OnHeap peak) before its 6 h decommission. Exec-3 was the lone surviving workhorse — it shut down with the driver at 00:02:42, not before. Stages tab reports **5 058 completed stages** (UI capped to 958 shown) and 40 skipped — consistent with 1 283 bronze + 128 silver micro-batches over 10 h. Total executor task time 14.9 h across 10 h 8 m wall clock = **~1.5 average active executors** versus 4 requested — capacity was degraded the entire run.

Executor stderr — cause-of-death per Dead executor

Logs pulled from s3://emr-migration-poc/emr-eks-logs/brtw9ztup2y01wok6x8r2bsmr/jobs/000000037h6oker0s23/containers/spark-000000037h6oker0s23/. All three Dead executors logged identical clean-drain sequences; only exec-7 (a replacement spawned at end-of-run) failed to bootstrap.

Executor 4 (10.70.24.54) — lifetime ~14 s

```
26/05/14 14:08:05 INFO Executor: Adding file:/tmp/./...findbugs-jsr305-3.0.0.jar to class loader default
26/05/14 14:08:08 INFO BlockManager: Removing RDD 666, 626, 666
26/05/14 14:08:20 INFO CoarseGrainedExecutorBackend: Decommission executor 4.
26/05/14 14:08:20 INFO CoarseGrainedExecutorBackend: Will exit when finished decommissioning
26/05/14 14:08:20 INFO BlockManagerDecommissioner: Attempting to migrate all RDD blocks
26/05/14 14:08:20 WARN BlockManagerDecommissioner: Asked to decommission RDD cache blocks, but no blocks to migrate
26/05/14 14:08:21 INFO CoarseGrainedExecutorBackend: No running tasks, all blocks migrated, stopping.
26/05/14 14:08:21 INFO CoarseGrainedExecutorBackend: Executor self-exiting due to : Finished decommissioning
26/05/14 14:08:21 INFO ShutdownHookManager: Shutdown hook called
```

Diagnosis: Spot reclaim within ~15 s of pod readiness. Karpenter received the Spot interruption notice before the executor had picked up any tasks — 'no blocks to migrate'.

Executor 1 (10.70.31.49) — lifetime ~6 h 14 m

```
26/05/14 20:09:41 driver-stderr: Received executor 1 decommissioned message
26/05/14 20:10:12 INFO ConsumerCoordinator: consumer pro-actively leaving the group (× 5 Kafka consumers)
26/05/14 20:10:13 INFO ShutdownHookManager: Shutdown hook called
26/05/14 20:10:13 INFO CoarseGrainedExecutorBackend: No running tasks, all blocks migrated, stopping.
26/05/14 20:10:15 INFO MetricsSystemImpl: s3a-file-system metrics system shutdown complete.
```

Diagnosis: Spot reclaim with a clean ~30 s drain window. All shuffle blocks migrated successfully to peer executors and the s3a fallback bucket. Driver's next batch then spent 164.7 s rebuilding work — the decommission was clean, the recovery was not.

Executor 2 (10.70.22.93) — lifetime ~7 h 30 m

```
26/05/14 21:25:37 driver-stderr: Received executor 2 decommissioned message
26/05/14 21:26:08 INFO ConsumerCoordinator: consumer pro-actively leaving the group (× 5 Kafka consumers)
26/05/14 21:26:08 INFO ShutdownHookManager: Shutdown hook called
26/05/14 21:26:09 INFO CoarseGrainedExecutorBackend: No running tasks, all blocks migrated, stopping.
26/05/14 21:26:10 INFO MetricsSystemImpl: s3a-file-system metrics system shutdown complete.
```

Diagnosis: Identical Spot-reclaim signature to exec-1. ~31 s drain. Driver's next batch spent 165.4 s.

Executor 7 (replacement, never reached steady state)

```
OpenJDK 64-Bit Server VM warning: UseAVX=2 is not supported on this CPU, setting it to UseAVX=1
[no further log lines – stderr is 107 bytes total]
```

Diagnosis: Replacement pod was spawned by Karpenter to backfill capacity after the earlier decommissions, but its JVM only logged the AVX warning before the driver shutdown sequence terminated it. Not a failure cause; an artifact of the driver exit.

Why this run failed faster than the 29 h run

The silver-cadence change worked. The 1.0 → 5.5 MiB *task of very large size* ramp that defined the previous run is absent. Silver fired only every 10th bronze batch (last silver = Batch 1280); MERGE planning stayed flat. The metadata-bloat surface we engineered against is no longer the binding constraint.

The Spot-executor change opened a new failure surface. Three executor decommissions in 10 h on R-series Spot in ap-south-1a. Each cost a 2 m 45 s batch stall; in steady state the driver was tracking shuffle-migration completions, fallback-storage writes, and replacement-pod scheduling on top of normal batch work. With `spark.kubernetes.executor.deleteOnTermination=false` (kept for log inspection), dead executor pods remain registered with the driver — every Spot interruption adds permanent metadata to the driver heap rather than freeing it.

Net trade. We replaced one slow-burn driver-pressure curve (metadata bloat over 29 h) with a faster-burn one (Spot churn + decommission bookkeeping over 10 h). The terminal symptom is identical: external SIGTERM to a driver pod that still has clean shutdown-hook output. We still cannot directly observe the kubelet eviction event (no Container Insights, no Grafana access, no kubectl).

Next-run levers (keep Spot, diversify it)

Spot stays. The fix is to make Spot reclaim less frequent and less synchronised by widening the instance pool Karpenter can draw from.

- **Move executors to ARM64 / Graviton Spot.** Set

`spark.kubernetes.executor.node.selector.kubernetes.io/arch: arm64` in the submission JSON and update `executor-podtemplate-spot.yaml` to match. Graviton Spot (m6g / m7g / r6g / r7g) typically has **30–50 % lower reclaim rates** than the equivalent x86 Spot pool in ap-south-1, and the Spot price itself is ~20 % cheaper — strictly better on both the stability and cost axes. EMR-on-EKS 7.9.0 ships multi-arch images (`spark/emr-7.9.0:latest`), so the only change is the node selector. Driver can stay on x86 on-demand for now to minimise variables.

- **Broaden the executor instance-category from r to [m, r].** Today the submission pins `karpenter.k8s.aws/instance-category: r`. The NodePool already allows `m / r / t` — removing the pin (or switching it to a `NotIn: [t]` requirement at the pod-template level) lets Karpenter pick the price-capacity-optimised Spot pool across both M and R families. More diverse pools = lower chance of all executors getting reclaimed in the same Spot-rebalancing wave (which is exactly what we saw: 3 reclaims in 10 h on a single R-only Spot pool in one AZ).

- **Promote the driver pod to Guaranteed QoS.** Today the driver is Burstable (memory request 10 G ≠ limit 12 G), the QoS tier kubelet evicts *first* under host memory pressure. Set `spark.kubernetes.driver.limit.memory: 10G` (= request) in the submission JSON. Guaranteed-class pods are the last to be evicted; if a future failure still fires, it will show up as a container-OOM (exit 137 / SIGKILL, no shutdown hooks) rather than exit 143, which is diagnostically distinct from kubelet eviction.

- **Flip** `spark.kubernetes.executor.deleteOnTermination=true`. Currently false to keep dead pods around for inspection — but with Spot we should expect frequent decommissions, and every dead pod that lingers adds bookkeeping to the driver heap. Logs already ship to S3, so flipping this loses nothing real.

- **Add an S3 heap-dump on driver SIGTERM.** `-XX:HeapDumpOnOutOfMemoryError` does not fire on SIGTERM. Add a Python signal handler in `streaming_v3_s3tables_eks.py` that on SIGTERM shells out to `jmap -dump` and uploads the hprof to `s3://emr-migration-poc/heapdumps/<job-id>/.` Next eviction leaves driver-heap evidence behind.
- **Hold** `SILVER_EVERY_N_BATCHES = 10`. The cadence change held for 10 h with no metadata-bloat curve. Leave it alone.